PRODUCTION DATASHEET: LOTUS V3-ULTRA

Architect: Rudolf Juhl | **Location:** Bensheim, Germany
Classification: Open Science Property | **Scale:** 300 Qubits (6-Die Package)

1. System Architecture & Absolute Maximum Ratings

Symbol	Description	Min	Max	Unit
Vdd_max	Core Transistor Voltage Window	-0.3	0.92	V
T_junc	Maximum Silicon Junction Temperature	-40	+85.0	°C
V_esd	Electrostatic Discharge Protection (HBM)	-	2000	V

2. Physical Dimensions & Pin Mapping

- **Lithography Node:** TSMC N2 (2-Nanometer Gate-All-Around EUV).
- **Package Footprint:** 45.0 mm × 45.0 mm Substrate Carrier using CoWoS-S.
- **Power Input Matrix:** 925 Pins Vdd Core (0.78V) / 925 Pins Vss Ground. Max load 250 mA per bump.
- **Bus Interconnect:** 1024-Bit Parallel Lanes mapped for 5.50 GHz Inter-Die Ring-Bus.

3. SRAM Memory Address Map

Die Segment	Qubit Range	Start Address	End Address
Die 01 / Die 02	Q001 - Q100	`0x0000_0000`	`0x0000_07CF`
Die 03 / Die 04	Q101 - Q200	`0x0000_07D0`	`0x0000_0F9F`
Die 05 / Die 06	Q201 - Q300	`0x0000_0FA0`	`0x0000_176F`

> **LOTUS V3-ULTRA CIVILIAN USE DIRECTIVE:** Absolute Civilian Use Only. Any military deployment, defense-sector integration, or weaponization is strictly prohibited. This platform completely rejects any calculations related to fissile materials or weapons of mass destruction.

4. Hardware Description Language (HDL) Source Code

```
// ============================================================================
// HARDWARE ARCHITECTURE NETLIST: LOTUS V3-ULTRA (CIVILIAN CORE ONLY)
// Technology Node: 2nm Gate-All-Around (GAA) EUV Lithography
// Top-Level Module: lotus_v3_ultra_top
// ============================================================================

module lotus_v3_ultra_top (
    input  wire         clk_global,         // 5.50 GHz Symmetrical Master Clock
    input  wire         rst_n,              // Active-Low Hardware Reset
```

```
    input  wire [1023:0] ext_civil_data_in,  // 1024-Bit Civil Sensor/Data Inward Array
    output wire [1023:0] ext_civil_data_out, // 1024-Bit Processed Output Array
    output wire          system_stable_flag  // Global Hardware Health Status
);

    // ------------------------------------------------------------------------
    // INTERNAL POWER AND INTERCONNECT WIRE MATRICES (1024-BIT BUS INTERCONNECT)
    // ------------------------------------------------------------------------
    wire [1023:0] ring_bus_01_02;
    wire [1023:0] ring_bus_02_03;
    wire [1023:0] ring_bus_03_04;
    wire [1023:0] ring_bus_04_05;
    wire [1023:0] ring_bus_05_06;
    wire [1023:0] ring_bus_06_01;

    wire [5:0]    die_status_vector;
    assign system_stable_flag = &die_status_vector;

    // ------------------------------------------------------------------------
    // INSTANTIATION OF THE 6 INDEPENDENT HARDWARE DIES (50 QUBITS PER DIE)
    // ------------------------------------------------------------------------

    // DIE 01 - Quantum Emulation Register Block 01-50
    lotus_compute_die die_01 (
        .clk(clk_global), .rst_n(rst_n),
        .bus_in(ring_bus_06_01), .bus_out(ring_bus_01_02),
        .data_io_slice(ext_civil_data_in[169:0]),
        .die_ready(die_status_vector[0])
    );

    // DIE 02 - Quantum Emulation Register Block 51-100
    lotus_compute_die die_02 (
        .clk(clk_global), .rst_n(rst_n),
        .bus_in(ring_bus_01_02), .bus_out(ring_bus_02_03),
        .data_io_slice(ext_civil_data_in[339:170]),
        .die_ready(die_status_vector[1])
    );

    // DIE 03 - Quantum Emulation Register Block 101-150
    lotus_compute_die die_03 (
        .clk(clk_global), .rst_n(rst_n),
        .bus_in(ring_bus_02_03), .bus_out(ring_bus_03_04),
        .data_io_slice(ext_civil_data_in[509:340]),
        .die_ready(die_status_vector[2])
    );

    // DIE 04 - Quantum Emulation Register Block 151-200
    lotus_compute_die die_04 (
        .clk(clk_global), .rst_n(rst_n),
        .bus_in(ring_bus_03_04), .bus_out(ring_bus_04_05),
        .data_io_slice(ext_civil_data_in[679:510]),
        .die_ready(die_status_vector[3])
    );

    // DIE 05 - Quantum Emulation Register Block 201-250
    lotus_compute_die die_05 (
        .clk(clk_global), .rst_n(rst_n),
        .bus_in(ring_bus_04_05), .bus_out(ring_bus_05_06),
        .data_io_slice(ext_civil_data_in[849:680]),
        .die_ready(die_status_vector[4])
```

```
    );

    // DIE 06 - Quantum Emulation Register Block 251-300
    lotus_compute_die die_06 (
        .clk(clk_global), .rst_n(rst_n),
        .bus_in(ring_bus_05_06), .bus_out(ring_bus_06_01),
        .data_io_slice(ext_civil_data_in[1019:850]),
        .die_ready(die_status_vector[5])
    );

    // Output Mapping Matrix
    assign ext_civil_data_out = ring_bus_01_02 ^ ring_bus_04_05;

endmodule

// ============================================================================
// HARDWARE INTRINSIC CORE: COMPUTE DIE DESIGN (50 QUBITS STATE EMULATION)
// ============================================================================
module lotus_compute_die (
    input  wire          clk,
    input  wire          rst_n,
    input  wire [1023:0] bus_in,
    output reg  [1023:0] bus_out,
    input  wire [169:0]  data_io_slice,
    output wire          die_ready
);
    // Internal Register Array Declarations removed for behavioral abstraction
    reg [15:0] phase_correction_register;
    assign die_ready = rst_n; // Hardware Flag Status Anchor

    // ----------------------------------------------------------------------
    // HARDWARE BLOCK: PARACONSISTENT TENSOR ROUTER & VIRTUAL-Z INJECTION
    // ----------------------------------------------------------------------
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            bus_out <= 1024'b0;
            phase_correction_register <= 16'h0000;
        end else begin
            // 1. Virtual-Z Correction轉 (Phase-Skew Error Elimination)
            if (bus_in[7:0] == 8'hFF) begin
                phase_correction_register <= phase_correction_register + 16'h0005; // Auto-
Phase Align
                bus_out <= bus_in << 2; // Instantaneous Zero-Delay Phase Shift
            end else begin
                // 2. Paraconsistent Processing Routing Step
                bus_out <= (bus_in ^ {854'b0, data_io_slice}) + 1024'hA5A5;
            end
        end
    end
endmodule
```